

CONCEPTION D'UN PROCESSEUR 8 BITS

2025-2026

Auteurs :

BE

OD

Introduction

Ce rapport présente l'intégralité du travail mené pour concevoir un processeur 8 bits sur Logisim Evolution. Le projet a été réalisé étape par étape, nous avons commencé des portes logiques jusqu'à atteindre un CPU complet.

Le fichier cpu8.circ regroupe 17 circuits imbriqués. Plutôt que de suivre l'ordre chronologique des étapes, ce rapport décrit chaque sous-circuit du plus bas niveau jusqu'au plus haut niveau. Chaque section précisera le fonctionnement et un scénario de test avec les résultats observés.

Le jeu d'instructions que nous avons défini est encodé sur 14 bits et supporte l'affectation d'une valeur dans un registre, les opérations registre-registre (ADD, AND, OR, XOR), le chargement depuis la RAM, le stockage dans ce dernier et le saut incondtionnel (JUMP). Les registres R0 à R3 manipulent les données sur 8 bits non signés.

1. Unités logiques bit à bit

1.1-AND_8_bits

Ce sous-circuit réalise un ET logique bit à bit entre deux bus (broche / pin) de 8 bits. C'est le premier module que nous avons conçu, sans aucun composant de haut niveau, uniquement avec des portes AND à 2 entrées de 1 bit.

L'architecture : 2 Splitter décomposent chacun des bus d'entrée en 8 fils individuels (un par bit). 8 portes AND à 2 entrées calculent ensuite chaque bit de la sortie en parallèle. Un Splitter de reconstruction rassemble les 8 résultats en un bus de sortie de 8 bits.

Interface : Entrée 1 (8 bits) et Entrée 2 (8 bits) | Sortie (8 bits)

Composants : 3 Splitter de 8 bits + 8 porte AND 1 bit

Test & Vérification

Entrée 1 = 0b101110011
Entrée 2 = 0b11001010

Résultat observé = 0b10000010

1.2-OR_8_bits

Même architecture que le AND, en substituant les portes AND par des portes OR. 2 Splitter en entrée, 8 portes OR 1 bit, 1 Splitter en sortie.

Interface : Entrée 1 (8 bits) et Entrée 2 (8 bits) | Sortie (8 bits)

Test et Vérification

Entrée 1 = 0b101110011
Entrée 2 = 0b11001010

Résultat : 0b11111011

1.2-XOR_8_bits

Même logique, portes XOR cette fois. Ce module sera utilisé à la fois dans l'additionneur 1 bit (calcul de la somme) et dans l'UAL pour l'opération XOR registre à registre.

Interface : Entrée 1 (8 bits) et Entrée 2 (8 bits) | Sortie (8 bits)

Test & Vérification

Entrée 1 = 0b101110011
Entrée 2 = 0b11001010

Résultat observé : 0b01111001

1.4-UL_setp_2 – Unité Logique

Ce circuit réunit les trois modules précédents sous une interface unique. Il a deux entrées de 8 bits (**A** et **B**) et produit simultanément et en permanence trois sorties : **A AND B**, **A OR B** et **A XOR B**. Ce n'est pas encore l'UAL finale : ce module calcule les trois opérations en parallèle et laisse au niveau supérieur le soin de sélectionner le résultat utile via un multiplexeur.

Interface : A 8 bits (entrée) , B 8 bits (entrée) | AND, OR, XOR tous de 8 bits (sorties)

Test & Vérification

Entrée : A = 0x0F et B = 0xF0

Sortie : AND = 0x00

OR = 0xFF

XOR = 0xFF

2.Additionneur 8 bits

2.1-Additionneur_1_bit

Avant de construire l'additionneur 8 bits, nous avons conçu la base : un **additionneur 1 bit** avec retenue entrante. Ce module possède trois entrées d'1 bit chacune (a, b, Rin) et deux sorties d'1 bit (somme, Rout).

Nous avons dérivé les équations booléennes de la table de vérité. La forme normale disjonctive donne :

$$\text{Somme} = (a \text{ XOR } b) \text{ XOR } \text{Rin}$$

$$\text{Rout} = (a \text{ AND } b) \text{ OR } (\text{Rin AND } (a \text{ XOR } b))$$

L'implémentation dans Logisim utilise uniquement des portes à 2 entrées 1 bit. On compte au total 4 portes XOR, 4 portes AND, 1 porte OR et 4 portes NOT (pour produire les compléments nécessaires à la forme FND complète avant simplification).

Interface : a : 1 bit, b : 1 bit, Rin : 1 bit | somme : 1 bit, Rout : 1 bit

Rin	a	b	somme	Rout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.2-Additionneur_8_bits

L'additionneur 8 bits est construit par mise en cascade de **8 instances** de l'ADDITIONNEUR_1_bit. La retenue sortante d'un étage alimente la retenue entrante de l'étage suivant, du bit de poids faible (bit 0) vers le bit de poids fort (bit 7).

Deux Splitter décomposent les bus d'entrée (8 bits chacun) en fils individuels. Un troisième Splitter reconstruit le bus de sortie de 8 bits à partir des 8 sommes partielles. La retenue initiale (Rin de l'étage 0) est câblée à 0.

Interface : a : 8 bits, b : 8 bits, Rin : 1 bit | somme : 8 bits, Rout : 1 bit

Test & Vérification

a = 0x05 (5), b = 0x00 (0), Rin = 0 -> somme = 0x05, Rout = 0

a = 0xFF (255), b = 0x01 (1), Rin = 0 -> somme = 0x00, Rout = 1

a = 0x0A (10), b = 0x05 (5), Rin = 0 -> somme = 0x0F (15)

[Il existe une limite (connue) pour un additionneur à propagation de retenue, les délais s'accumulent sur 8 niveaux.]

3. Unité Arithmétique et Logique (UAL_8_BITS)

L'UAL est le cœur d'opération du processeur. Elle prend en entrée deux opérandes de 8 bits (A et B) et un signal de sélection d'opération sur 2 bits (OP), et produit un résultat de 8 bits. Les quatre opérations possibles sont :

OP (bits 12-11)	Opération	Résultat
00	ADD	A + B (8 bits)
01	AND	A AND B
10	OR	A OR B
11	XOR	A XOR B

D'un point de vue architecture, l'UAL instancie simultanément **tous les sous-modules** : ADDITIONNEUR_8_bits, AND_8_bits, OR_8_bits, XOR_8_bits. Les quatre résultats sont calculés en parallèle et présentés aux entrées d'un multiplexeur 4-vers-1 à sélecteur 2 bits. Le signal OP choisit lequel des quatre bus de 8 bits est routé vers la sortie.

Interface : A : 8 bits, B : 8 bits, OP : 2 bits | Résultat : 8 bits

N.B. : Le choix de calculer tous parallèlement (à chaque cycle) puis de choisir avec un multiplexeur (plutôt qu'un chemin séquentiel) simplifie le câblage et élimine tout problème entre opérations : la sortie est toujours stable après la propagation combinatoire des modules (i.e. pas d'horloge).

Test & Vérification

OP=00 : A=0x05, B=0x00 -> Résultat=0x05 (ADD)

OP=01 : A=0x05, B=0x00 -> Résultat=0x00 (AND)

OP=10 : A=0x00, B=0x05 -> Résultat=0x05 (OR)

OP=11 : A=0x05, B=0x05 -> Résultat=0x00 (XOR)

4. Banc de registres

4.1-Registre_8_bits – Registre individuel

Un registre 8 bits est construit à partir de **8 bascules D** câblées sur le même bus (pin) d'horloge. Chaque bascule D stocke 1 bit : sur le front montant de l'horloge, elle capture la valeur présente sur son entrée D et la maintient sur sa sortie Q jusqu'au prochain front.

Un signal **Write Enable (WE)** contrôle l'activation individuelle de chaque bascule (via leur entrée enable). Lorsque WE = 0, les bascules ignorent l'horloge : la valeur stockée est figée. Lorsque WE = 1, le front d'horloge charge la nouvelle valeur.

Deux Splitter encadrent les bascules : l'un décompose le bus d'entrée en 8 fils, l'autre reconstruit le bus de sortie depuis les 8 sorties Q.

Interface : Entrée : 8 bits, Clock : 1 bit, WE : 1 bit | Sortie Q : 8 bits

Test & Vérification

WE=1, Clock, Entrée=0x2A -> Q = 0x2A au cycle suivant

WE=0, Clock, Entrée=0xFF -> Q conserve 0x2A

4.2-Banc_4_Registres – Banc de quatre registres

Le banc de registres regroupe quatre instances de REGISTRE_8_bits (R0, R1, R2, R3). Il gère en parallèle : l'écriture dans le registre de destination et la lecture simultanée de deux registres sources, nécessaire pour alimenter les deux opérandes de l'UAL dans un même cycle.

Trois signaux de sélection de 2 bits chacun adressent les registres : **ADDR_RES** (registre de destination pour l'écriture), **ADDR_OP1** (premier opérande, sortie A) et **ADDR_OP2** (second opérande, sortie B).

L'écriture est orchestrée par 6 multiplexeurs 4-vers-1 : quatre mux génèrent les signaux WE individuels (un seul actif à la fois selon ADDR_RES + WE global) ; deux mux sélectionnent les valeurs de sortie sur les bus A et B selon les adresses OP1 et OP2.

Interface : E : 8 bits (donnée), ADDR_RES : 2 bits, ADDR_OP1 : 2 bits, ADDR_OP2 : 2 bits, WE : 1 bit, Clock | A : 8 bits, B : 8 bits

Test & Vérification

ADDR_OP1=01 (R1), ADDR_OP2=00 (R0) -> A=0x05, B=0x00

5. Unité de traitement – UAL_ET_BANC

Ce circuit assemble le banc de registres et l'UAL en boucle fermée : les sorties A et B du banc alimentent les opérandes de l'UAL, et le résultat de l'UAL est rebouclé sur l'entrée E du banc. C'est l'étape de validation fonctionnelle avant d'introduire la ROM.

Un **multiplexeur d'entrée** permet de choisir entre la boucle (résultat UAL) et une constante externe injectée manuellement (utile pour initialiser un registre à une valeur non nulle puisque tous les registres démarrent à 0x00 à la mise sous tension).

Les signaux de contrôle exposés à l'extérieur sont : le signal d'opération OP (2 bits), les adresses des deux opérandes, l'adresse de destination, le WE global et l'horloge. Ces signaux seront ensuite produits automatiquement par le décodeur d'instructions.

Ce que l'on observe :

Cycle 1 : on charge R1=5 via le multiplexeur externe

Cycle 2 : ADD R2 <- R1 + R0, OP = 00

Résultat 5 rebouclé dans R2.

Test & vérification

Injection : ADDR_RES=01, WE=1, MUX->constante=5, Clock -> R1=5

ADD : OP=00, ADDR_OP1=01, ADDR_OP2=00 -> UAL calcule 5+0=5

ADDR_RES=10, WE=1, MUX->UAL, Clock -> R2=5

XOR : OP=11, ADDR_OP1=01, ADDR_OP2=10 -> 5 XOR 5 = 0 -> R3=0

6.Mémoire programme et décodage d'instruction

6.1-Format d'instruction – 14 bits

Le format d'instruction sur 14 bits que nous avons conçu est le suivant. L'encodage a été pensé pour que le décodeur soit purement combinatoire, sans état interne.

Bit 13	Bits 12-11	Bits 10-9	Bits 8-1	Bit 0
MODE	OP	ADDR_RES	CHAMP VARIABLE	WE
1=immédiat	ADD/AND/OR/XOR	R0..R3	DATA (8b) ou ADDR_OP1+OP2	Write Enable

MODE=1 : affectation immédiate. Bits 8-1 = valeur sur 8 bits à charger dans ADDR_RES. OP est ignoré.

MODE=0 : opération registre à registre. Bits 6-5 = adresse source 1 (LSB), bits 2-1 = adresse source 2 (LSB). Bits 8-7 et 4-3 sont des don't-cares (mis à 00).

Les cinq instructions de test encodées dans la ROM sont :

Adresse	Instruction	14 bits	Hex
0x00	R1 = 5	10001000001011	220B
0x01	R2 = R1 + R0	00010000100001	0421
0x02	R3 = R1 XOR R2	01111000100101	1E25
0x03	R0 = R2 AND R3	00100001000111	0847
0x04	R1 = R0 OR R2	01001000000101	1205

6.2-Pointeur_programme – Compteur Programme

Le Pointeur Programme est un registre 8 bits qui se comporte comme un compteur : à chaque front d'horloge, il s'incrémente de 1 pour pointer vers l'instruction suivante. Il est câblé via une constante additive de 1 rebouclée sur son entrée de chargement. La valeur initiale au reset est 0x00, correspondant à la première instruction en ROM.

Interface : Clock : 1 bit, Reset : 1 bit | PC : 8 bits (adresse courante vers ROM)

Test & Vérification

Reset=1 -> PC = 0x00 immédiatement

Clock (x5) -> PC = 0x00 -> 0x01 -> 0x02 -> 0x03 -> 0x04 -> 0x05

6.3-Decodage_Instruction – Décodeur combinatoire

Le décodeur reçoit les 14 bits d'une instruction lue depuis la ROM et génère directement tous les signaux de contrôle nécessaires à l'unité de traitement (sans micro-programme ni ROM de contrôle). C'est un circuit combinatoire composé de Splitter, portes NOT, AND et d'une constante.

L'extraction des champs se fait par Splitter appliqués sur le bus de 14 bits :

- Bit 13 -> **MODE** (immédiat ou registre à registre)
- Bits 12-11 -> **OP** (opération UAL, 2 bits)
- Bits 10-9 -> **ADDR_RES** (registre destination, 2 bits)
- Bits 8-1 -> **DATA ou ADDR_OP1+OP2** (selon MODE)
- Bit 0 -> **WE** (write enable)

En mode immédiat (MODE=1), les 8 bits de DATA sont directement sur l'entrée E du banc de registres. Le signal de sélection du mux d'entrée est positionné sur la donnée immédiate plutôt que sur le résultat de l'UAL. En mode registre (MODE=0), les bits 6-5 et 2-1 fournissent respectivement ADDR_OP1 et ADDR_OP2.

Interface (entrées) : Instruction : 14 bits provenant de la ROM

Interface (sorties) : OP : 2 bits | ADDR_RES : 2 bits | ADDR_OP1 : 2 bits | ADDR_OP2 : 2 bits | WE : 1 bit | DATA : 8 bits | MODE : 1 bit

Test & Vérifications

Instruction=220B (0x220B) -> MODE=1, ADDR_RES=01 (R1), DATA=0x05, WE=1

-> Signal contrôle correctement décodé, R1 recevra 5

Instruction=0421 -> MODE=0, OP=00 (ADD), ADDR_RES=10 (R2), ADDR_OP1=01 (R1), ADDR_OP2=00 (R0), WE=1

6.4-Etape7_ROM – Processeur complet avec ROM

Ce circuit est le premier processeur fonctionnel complet : il intègre l'unité de traitement (UAL_ET_BANC), le décodeur d'instructions (DECODAGE_INSTRUCTION), la ROM, le pointeur programme (Pointeur_Programme), et la logique de multiplexage du bus de données.

Le flux d'exécution d'une instruction est le suivant :

- 1. Le PC fournit une adresse à la ROM.
- 2. La ROM sort les 14 bits de l'instruction correspondante.
- 3. Le décodeur analyse l'instruction et génère tous les signaux de contrôle.
- 4. L'unité de traitement exécute l'opération (UAL ou chargement immédiat).
- 5. Sur le front montant de l'horloge, le résultat est écrit dans le registre cible et le PC s'incrémente.

Un signal **AND** entre le WE provenant du décodeur et une constante 1 assure que l'écriture n'a lieu que si l'instruction le requiert.

Test & Vérification

Programme ROM : 220B 0421 1E25 0847 1205

Cycle 0 (PC=0x00) : R1 <- 5 (instruction 220B)

Cycle 1 (PC=0x01) : R2 <- R1+R0 = 5+0=5 (instruction 0421)

Cycle 2 (PC=0x02) : R3 <- R1 XOR R2=5 XOR 5=0 (instruction 1E25)

Cycle 3 (PC=0x03) : R0 <- R2 AND R3=5 AND 0=0 (instruction 0847)

Cycle 4 (PC=0x04) : R1 <- R0 OR R2=0 OR 5=5 (instruction 1205)

Résultats finaux : R0=0, R1=5, R2=5, R3=0

7.Extension RAM – ETAPE8_RAM

Pour dépasser la limitation des quatre registres, nous avons ajouté une **mémoire RAM** accessible en lecture (LOAD) et en écriture (STORE). Cette extension nécessite deux nouveaux types d'instructions et une unité LOAD/STORE intercalée entre le décodeur et l'unité de traitement.

Les trois nouvelles instructions supportées sont :

- LOAD Ri, imm
- LOAD Ri, @addr
- STORE @addr, Ri

Le circuit ETAPE8_RAM intègre : la ROM programme (inchangée), la RAM données, le PC, le décodeur (étendu avec les bits de type d'accès mémoire), et deux multiplexeurs. Le premier multiplexeur sélectionne la source des données pour l'écriture dans le banc de registres (résultat UAL ou lecture RAM). Le second multiplexeur sélectionne l'adresse RAM.

Donc : ROM | RAM | 2 mux | 1 Constante (addr initiale)

Test & Vérification

STORE @0x02, R1 : valeur de R1 écrite en mem[0x02]

-> RAM[0x02] = 0x05 si R1=5

LOAD R3, @0x02 : R3 <- mem[0x02] = 0x05

LOAD R0, 0x12 : R0 <- 0x12 (décimal 18) directement

8.Instructions de branchement – ETAPE9_JUMP

8.1-Pointeur_Programme_JMP

Le Program Counter (PC) original est un simple compteur +1. Pour supporter les sauts, nous l'avons étendu avec un multiplexeur 8 bits qui choisit, à chaque cycle, entre deux valeurs de prochain PC : PC+1 ou une adresse de saut fournie par le décodeur.

Un signal de contrôle **JUMP** (1 bit) sélectionne l'entrée du multiplexeur. Lorsque JUMP=0, le PC s'incrémente normalement. Lorsque JUMP=1, le PC charge l'adresse cible encodée dans l'instruction. La constante +1 et la constante d'adresse de JUMP sont toutes deux câblées sur le multiplexeur.

Interface : Clock | Reset | JUMP_ADDR : 8 bits | JUMP : 1 bit || PC : 8 bits

Test & Vérification

JUMP=0 : comportement compteur normal, PC++

JUMP=1, JUMP_ADDR=0x00 : PC <- 0x00 (retour au début du programme)

JUMP=1, JUMP_ADDR=0x03 : PC <- 0x03 (saut direct à l'adresse 3)

8.2-Decodage_Instruction_JUMP

Ce décodeur reprend l'intégralité du DECODAGE_INSTRUCTION et y ajoute la gestion du type JUMP. Un Splitter supplémentaire extrait les bits de type d'instruction. La logique AND/NOT combinatoire supplémentaire détecte JUMP et génère le signal JUMP envoyé au POINTEUR_PROGRAMME_JMP, ainsi que l'adresse cible extraite des bits de champ variable. Le bit de JUMP est là pour remplacer le bit de padding.

En présence d'une instruction JUMP, le signal WE est forcé à 0 (pas d'écriture en registre) et le signal JUMP=1 est émis. L'adresse cible est directement lue depuis les bits 2-9 de l'instruction (sur 8 bits), permettant de sauter à n'importe quelle adresse de la ROM (0x00 à 0xFF).

Donc : JUMP (1 bit) et JUMP_ADDR (8 bits)

8.3-ETAPE9_JUMP – Processeur complet avec JUMP

Ce circuit est le **processeur final** : il intègre tous les modules précédents (unité de traitement, ROM, RAM, décodeur JUMP, PC). Il est capable d'exécuter un programme avec boucles et sauts inconditionnels.

La RAM est toujours présente (mêmes LOAD/STORE qu'en étape 8). Le multiplexeur de données d'entrée du banc de registres supporte maintenant trois sources : résultat UAL, lecture RAM, et donnée directe, le décodeur pilote ce multiplexeur via ses bits de type d'instruction.

Le comportement sur un cycle avec JUMP est le suivant :

- Le PC fournit l'adresse de l'instruction JUMP à la ROM.
- Le décodeur détecte l'opcode JUMP, extrait l'adresse cible, émet JUMP=1 et WE=0.
- Le mux du PC sélectionne l'adresse cible au lieu de PC+1.
- Sur le front d'horloge, le PC charge la nouvelle adresse. Aucun registre n'est modifié.
- Au cycle suivant, l'exécution reprend à la nouvelle adresse.

Test & Vérification

Programme : 220B (R1←5), 0421 (R2←R1+R0), JUMP 0x00

Cycle 0 : R1=5

Cycle 1 : R2=5

Cycle 2 : JUMP -> PC <- 0x00 (boucle infinie)

Cycle 3 : R1 rechargé à 5 (boucle vérifiée sur pls cycles)

JUMP indirect (JUMP R1) : PC <- valeur de R1 = 5 -> exécution depuis 0x05

Conclusion

Nous sommes partis d'une porte AND 1-bit et avons abouti à un processeur 8 bits complet avec une mémoire programme (ROM), la mémoire de données (RAM) et le support des branchements inconditionnels. Chaque couche d'abstraction allant de la porte logique au module UAL, puis du décodeur au PC, a été construite et faite indépendamment avant d'être intégrée dans les niveaux supérieurs.

On pourrait améliorer ce projet par les points suivant :

- Ajouter des instructions de saut conditionnel (JUMP IF ZERO, JUMP IF CARRY) en exploitant les flags de l'UAL
- Étendre le banc à 8 registres
- Implémenter un pipeline simple